

EduMate - Vue d'ensemble du projet

Concept

EduMate est une plateforme de mise en relation pair-à-pair entre étudiants et tuteurs, enrichie par l'intelligence artificielle et sécurisée par la blockchain.

Vision du projet : Démocratiser l'accès au tutorat en permettant aux étudiants de partager leurs compétences et d'apprendre les uns des autres, tout en garantissant la traçabilité et l'équité des échanges via un système de crédits décentralisé (EduCoins).

Cas d'usage typique : Un étudiant en informatique peut proposer des cours de programmation en échange d'EduCoins, qu'il utilisera ensuite pour réserver des sessions de tutorat en mathématiques avec un autre étudiant. L'IA assiste à chaque étape : génération automatique d'annonces attractives, parsing de CV pour compléter le profil, et chatbot pour guider les utilisateurs.

Architecture

Le projet adopte une **architecture microservices** pour garantir la scalabilité, la maintenabilité et l'indépendance technologique de chaque composant. Chaque service communique via API REST ou WebSocket, permettant un développement et un déploiement indépendants.

Frontend (apps/)

- **web/** - Application React + Vite (port 5173) : interface utilisateur principale accessible via navigateur, avec gestion de l'état via Context API et appels API vers les microservices.
- **admin/** - Interface administrateur pour la gestion des utilisateurs, modération et statistiques.
- **mobile/** - Application mobile React Native (iOS/Android) pour l'accès nomade à la plateforme.

Backend Microservices (services/)

- **auth-service/** (Node.js, port 3001) - Gère l'authentification JWT, l'inscription, la connexion, les profils utilisateurs et les annonces de tutorat. C'est le service central qui orchestre les données utilisateur.
- **blockchain-service/** (Python/FastAPI, port 3003) - Interfaçage avec la blockchain Ethereum locale (Ganache). Gère les smart contracts EduToken et BookingEscrow pour les transactions de crédits et la sécurisation des paiements.
- **message-service/** (Node.js, port 3002) - Messagerie instantanée temps réel via WebSocket (Socket.io). Permet aux étudiants et tuteurs de communiquer directement.
- **cv-parser-service/** (Python/Flask, port 5001) - Extraction automatique d'informations structurées depuis un CV uploadé (PDF/DOCX) grâce à l'API Mistral AI.
- **booking-service/** - Gestion du cycle de vie des réservations : création, validation, annulation, et historique des sessions de tutorat.
- **chatbot-service/** - Agent conversationnel intelligent alimenté par OpenRouter (modèles Qwen/DeepSeek) pour assister les utilisateurs dans leurs recherches.
- **rag-service/** - Service de recherche sémantique avancée utilisant Qdrant (base vectorielle) pour recommander annonces et contenus pertinents.

Bases de données

- **PostgreSQL** (5432) - Base relationnelle principale : users, profiles, annonces, bookings, diplomas, experiences.
- **MongoDB** (27017) - Base NoSQL pour les données non structurées : messages, logs applicatifs, événements.

- **Qdrant** (6333) - Base vectorielle pour les embeddings et la recherche sémantique (IA).
- **Ganache** (8545) - Blockchain Ethereum locale pour développement et tests de smart contracts.

Démarrage rapide

```
# Avec Docker (recommandé)
docker-start.bat # Windows
./docker-start.sh # Linux/Mac

# Sans Docker
npm run dev
```

URLs principales:

- Frontend: <http://localhost:5173>
- Auth API: <http://localhost:3001>
- Blockchain: <http://localhost:3003>

Structure des fichiers clés

```
EduMate/
├─ APERCU_PROJET.md      # Fiche de synthèse du dépôt
├─ README.md             # Documentation générale du projet
├─ DOCKER_README.md      # Guide d'exécution Docker
├─ docker-compose.yml    # Orchestration des services
├─ Makefile              # Commandes utilitaires (build, up, logs, etc.)
├─ apps/
│   └─ web/              # Frontend principal (React + Vite)
│       ├── Dockerfile
│       ├── package.json
│       └─ src/
│           ├── components/ # UI réutilisable (modales, widgets, formulaires)
│           ├── pages/     # Pages métier (profile, booking, annonces...)
│           ├── services/  # Appels API (auth, profile, cv, bookings...)
│           ├── context/   # Context providers (session, état global)
│           ├── hooks/     # Hooks personnalisés
│           ├── types/     # Types TypeScript partagés
│           ├── utils/     # Fonctions utilitaires
│           └─ data/       # Données statiques et mocks
├─ services/
│   ├── auth-service/     # Authentification, profils, annonces
│   │   └─ src/
│   │       ├── routes/   # Endpoints REST
│   │       ├── controllers/ # Couche HTTP
│   │       ├── services/  # Logique métier
│   │       ├── models/   # Modèles BDD
│   │       └─ middlewares/ # Validation, sécurité, gestion erreurs
│   ├── message-service/  # Messagerie temps réel
│   ├── booking-service/  # Réservations et workflow sessions
│   ├── chatbot-service/  # Agent conversationnel IA
│   └─ cv-parser-service/ # Parsing CV via IA (Python)
```

```

|   ├── rag-service/          # Recherche sémantique/vectorielle
|   └── blockchain-service/   # Smart contracts et intégration Web3
|       ├── app/              # API Python blockchain
|       ├── contracts/        # Contrats Solidity
|       └── scripts/          # Déploiement et initialisation wallets
└── docs/
    └── MAINTENANCE_REPORT.md # Rapport d'analyse et maintenance
└── ganache_data/             # Données locales blockchain (dev)
└── package.json              # Scripts racine (monorepo/workspace)

```

Points de repère pour un nouveau développeur :

- Le point d'entrée frontend est `apps/web/src/main.tsx` , puis `App.tsx` .
- Les appels API frontend sont centralisés dans `apps/web/src/services/` .
- Chaque microservice backend suit globalement la structure `routes -> controllers -> services -> models` .
- L'orchestration locale multi-services passe par `docker-compose.yml` .
- Les détails d'exploitation et de maintenance sont dans `docs/MAINTENANCE_REPORT.md` .

Fonctionnalités principales

Utilisateurs

Profils complets - Chaque utilisateur (étudiant ou tuteur) dispose d'un profil détaillé incluant ses compétences à enseigner, ses besoins d'apprentissage, ses diplômes, expériences, disponibilités et localisation. L'interface ([ProfilePage.tsx](#)) permet une personnalisation complète.

Authentification sécurisée - Système d'authentification robuste basé sur JWT avec possibilité d'activer la double authentification (2FA). Le service ([authService.ts](#)) gère les sessions utilisateur et la persistance des tokens.

Upload et parsing CV - Les utilisateurs peuvent uploader leur CV (PDF/DOCX) qui sera automatiquement analysé par l'IA Mistral pour extraire compétences, formations et expériences, pré-remplissant ainsi le profil ([cvService.ts](#)).

Annonces & Réservations

Création d'annonces assistée par IA - Les tuteurs créent des annonces de tutorat. L'IA ([aiTextProcessor.js](#)) génère automatiquement des descriptions attractives et optimisées à partir de mots-clés simples, améliorant la visibilité et la qualité des annonces.

Système de réservation - Interface de booking ([BookingPage.tsx](#)) permettant de sélectionner créneaux disponibles, durée et mode (présentiel/distanciel). Le workflow inclut validation, confirmation et rappels.

Paiements décentralisés EduCoins - Toutes les transactions passent par des smart contracts ([blockchain-service](#)) garantissant traçabilité et équité. Les EduCoins sont bloqués en escrow pendant la session et libérés après validation.

Intelligence Artificielle

Génération de contenu - Utilisation de l'API OpenRouter avec modèles Qwen 2.5 et DeepSeek pour générer descriptions d'annonces, suggestions de compétences, et réponses contextuelles.

Parsing intelligent de CV - L'API Mistral ([mistral_service.py](#)) extrait automatiquement les sections Education, Experience, Skills depuis des documents non structurés.

Chatbot assistant - Widget conversationnel ([ChatbotWidget.tsx](#)) intégré dans toutes les pages pour répondre aux questions fréquentes, guider la navigation et recommander annonces pertinentes.

Stack technique

Couche	Technologies
Frontend	React 18, TypeScript, Vite, CSS Modules
Backend	Node.js 20, Express, FastAPI, Flask
BDD	PostgreSQL 16, MongoDB 7, Qdrant
Blockchain	Solidity, Web3.py, Ganache
DevOps	Docker, Docker Compose, Nginx
IA	OpenRouter, Mistral AI, RAG

Données et entités principales

Le modèle de données relationnel (PostgreSQL) est organisé autour de l'utilisateur (`User`) et de ses interactions (annonces, réservations, profils). Les relations clés :

- Un `User` peut avoir plusieurs `Annonces` (relation 1:N)
- Un `User` peut créer plusieurs `Bookings` en tant qu'étudiant ou recevoir des réservations en tant que tuteur (relation M:N)
- Un `ProfileData` est lié à un `User` (relation 1:1)

User (PostgreSQL)

```
interface User {
  id: number;
  email: string;
  password: string; // hashé avec bcrypt
  role: 'student' | 'tutor'; // un utilisateur peut être les deux
  profileComplete: boolean; // indique si le profil est rempli à 100%
  createdAt: Date;
  updatedAt: Date;
}
```

Annonce (PostgreSQL)

```
interface Annonce {
  id: number;
  tutorId: number; // foreign key vers User
  title: string;
  description: string; // peut être généré par IA
  subjects: string[]; // ex: ['Mathématiques', 'Physique']
  hourlyRate: number; // prix en EduCoins par heure
  teachingMode: 'online' | 'in-person' | 'both';
}
```

```
availability: string; // JSON des créneaux disponibles
active: boolean;
createdAt: Date;
}
```

ProfileData ([profileService.ts](#))

Données complètes du profil utilisateur :

- **Informations personnelles** - Nom, prénom, téléphone, photo de profil
- **Compétences** - Liste des compétences à enseigner (pour tuteurs) et à apprendre (pour étudiants)
- **Éducation** - Diplômes, établissements, années d'obtention
- **Expériences** - Expériences professionnelles et de tutorat
- **Disponibilités** - Plages horaires disponibles par jour de la semaine
- **Localisation** - Ville, code postal, rayon d'intervention (pour présentiel)

Booking (PostgreSQL)

```
interface Booking {
  id: number;
  studentId: number; // foreign key vers User
  tutorId: number; // foreign key vers User
  annonceId: number; // foreign key vers Annonce
  startTime: Date;
  endTime: Date;
  status: 'pending' | 'confirmed' | 'completed' | 'cancelled';
  transactionHash: string; // hash de la transaction blockchain
  amountEduCoins: number;
}
```

Workflow typique utilisateur

Parcours étudiant cherchant un tuteur :

1. **Inscription & Profil** - Création de compte → Upload CV (parsing auto) → Complétion profil (compétences à apprendre)
2. **Recherche** - Navigation annonces ou recherche sémantique (RAG) → Filtrage par matière, prix, mode, disponibilité
3. **Réservation** - Sélection annonce → Choix créneau → Paiement EduCoins (escrow blockchain) → Confirmation
4. **Communication** - Chat temps réel avec tuteur via message-service pour coordonner la session
5. **Session** - Rencontre physique ou visio → Validation post-session → Libération des EduCoins au tuteur
6. **Feedback** - Notation et avis (optionnel)

Parcours tuteur proposant ses services :

1. **Inscription & Profil** - Création compte → Upload CV → Complétion profil (compétences à enseigner)
2. **Création annonce** - Saisie mots-clés → Génération description par IA → Définition prix, mode, disponibilités
3. **Réception réservations** - Notification de nouvelles demandes → Validation/refus booking
4. **Communication** - Échange avec étudiant via chat
5. **Session** - Dispensation du cours → Validation post-session → Réception EduCoins

6. **Historique** - Consultation de l'historique des sessions et des gains via blockchain

Configuration développement

Variables d'environnement (.env)

Le fichier `.env` à la racine du projet configure tous les services. Exemple de configuration locale :

```
# PostgreSQL
POSTGRES_USER=edumate_user
POSTGRES_PASSWORD=edumate_password
POSTGRES_DB=edumate

# MongoDB
MONGO_INITDB_ROOT_USERNAME=admin
MONGO_INITDB_ROOT_PASSWORD=admin_password

# Blockchain
WEB3_PROVIDER_URL=http://ganache:8545
PRIVATE_KEY=0x4f3edf983ac636a65a842ce7c78d9aa706d3b113bce9c46f30d7d21715b23b1d

# IA Services
MISTRAL_API_KEY=your_mistral_key
OPENROUTER_API_KEY=sk-or-v1-...
```

Commandes utiles

Gestion des services Docker :

```
# Démarrer tous les services
docker-compose up -d

# Logs en temps réel d'un service spécifique
docker-compose logs -f auth-service

# Rebuild et redémarrer un service après modification du code
docker-compose up -d --build auth-service

# Arrêter tous les services
docker-compose down

# Supprimer volumes (réinitialiser BDD)
docker-compose down -v
```

Accès aux bases de données :

```
# Accès PostgreSQL (console interactive)
docker exec -it edumate-postgres psql -U edumate_user -d edumate

# Accès MongoDB (shell)
docker exec -it edumate-mongo mongosh -u admin -p admin_password
```

```
# Vérifier données Ganache (blockchain)
curl http://localhost:8545 -X POST --data
'{"jsonrpc":"2.0","method":"eth_accounts","params":[],"id":1}'
```

Health checks et debugging :

```
# Vérifier état auth-service
curl http://localhost:3001/health

# Vérifier état blockchain-service
curl http://localhost:3003/health

# Lister tous les conteneurs actifs
docker ps

# Inspecter les variables d'environnement d'un service
docker exec auth-service env
```

Documentation complète

- [README.md](#) - Vue d'ensemble et démarrage
- [DOCKER README.md](#) - Guide Docker complet (10 méthodes de lancement)
- [MAINTENANCE REPORT.md](#) - Analyse technique approfondie
- [Makefile](#) - Commandes simplifiées (`make start` , `make logs`)

Contribution

Convention commits

```
feat: ajout endpoint /api/bookings
fix: correction timeout blockchain
docs: mise à jour README
refactor: simplification auth middleware
test: ajout tests unitaires booking
```

Workflow

1. Fork le projet
2. Créer une branche (`git checkout -b feature/nouvelle-fonctionnalite`)
3. Commit (`git commit -m 'feat: ajout nouvelle fonctionnalité'`)
4. Push (`git push origin feature/nouvelle-fonctionnalite`)
5. Ouvrir une Pull Request

Support

- **Issues GitHub** - Bugs et suggestions
- **Email** - hasmir.boinali@etu.u-pec.fr & nthanhuyen1411@gmail.com
- **Documentation** - Consultez [MAINTENANCE REPORT.md](#) pour l'architecture détaillée

Auteurs: Équipe EduMate UPEC (Hasmir BOINALI & Thanh Uyen NGUYEN)